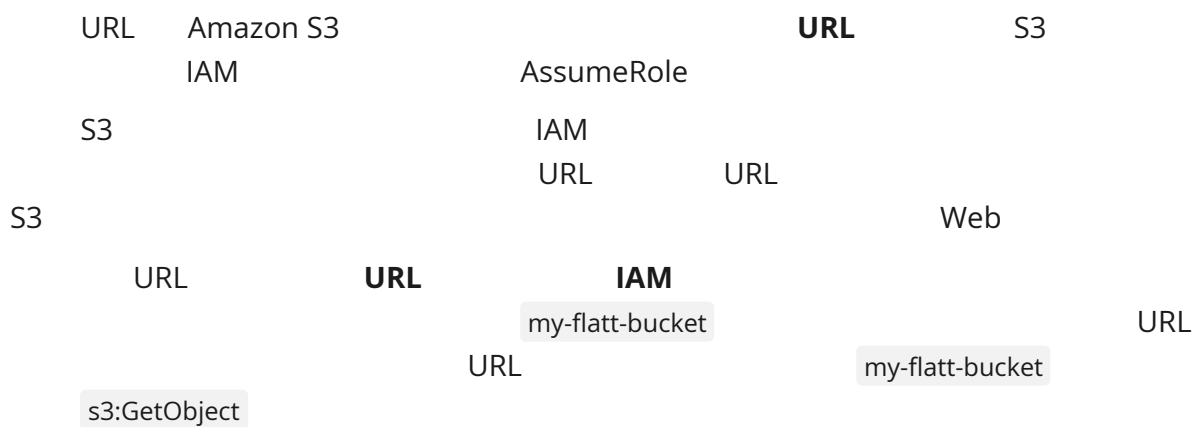


Path Traversal in Signed URLs, Which Even Existed in the Official AWS SDK

- []()
- []()
- [URL]()
- [URL]()

- [URL]()
- [S3]()
- [URL]()
- [AWS SDK]()
- [Go(v1) S3 URL]()
- []()
- []()
- [AWS]()
- []()
- []()
- [JavaScript(v3) CloudFront URL]()
- []()
- []()
- [AWS]()
- []()
- []()
- []()
- []()
- []()
- [URL]()
- []()
- [GMO Flatt Security]()

URL



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowDownloadFromMyFlattBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::my-flatt-bucket/*"
      ]
    }
  ]
}
```

Timeline of AWS S3 and its security features:

- 2006: S3, URL
- 2007: 2, 1
- 2008: URL
- 2009: 2
- 2010: AWS
- 2011: URL
- 2012: 2
- 2020: 2020 9 30
- 2021: SSL/TLS

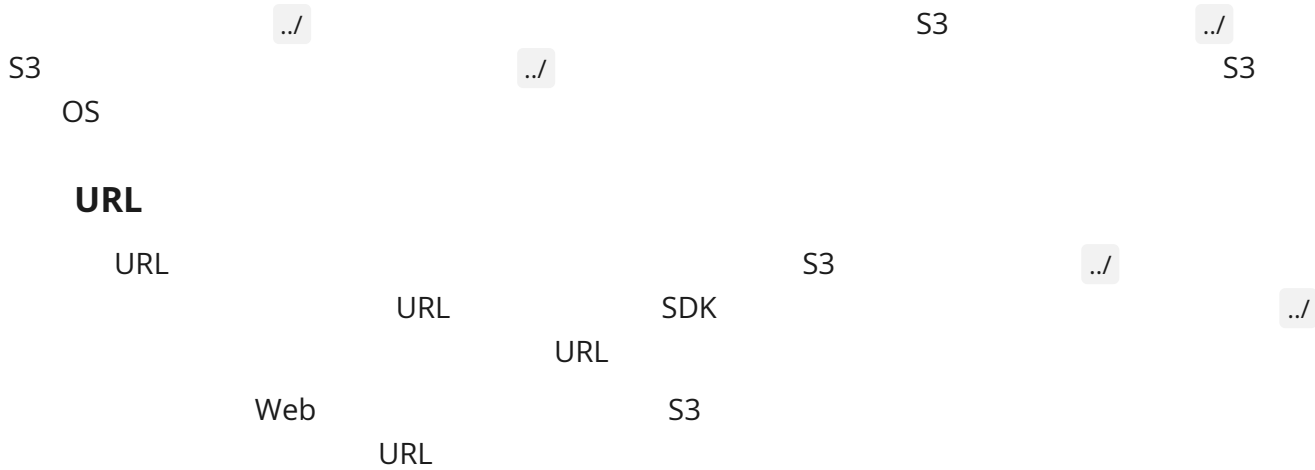
URL

```
| | | | | https://s3.${region-code}.amazonaws.com/${bucket-name}/${object-key} | | |  
| https://${bucket-name}.s3.${region-code}.amazonaws.com/${object-key} | |
```

URL _____

In Amazon S3 general purpose buckets, objects are the primary resources, and objects are stored in buckets. Amazon S3 general purpose buckets have a flat structure instead of a hierarchy like you would see in a file system. However, for the sake of organizational simplicity, the Amazon S3 console supports the folder concept as a means of grouping objects. Amazon S3

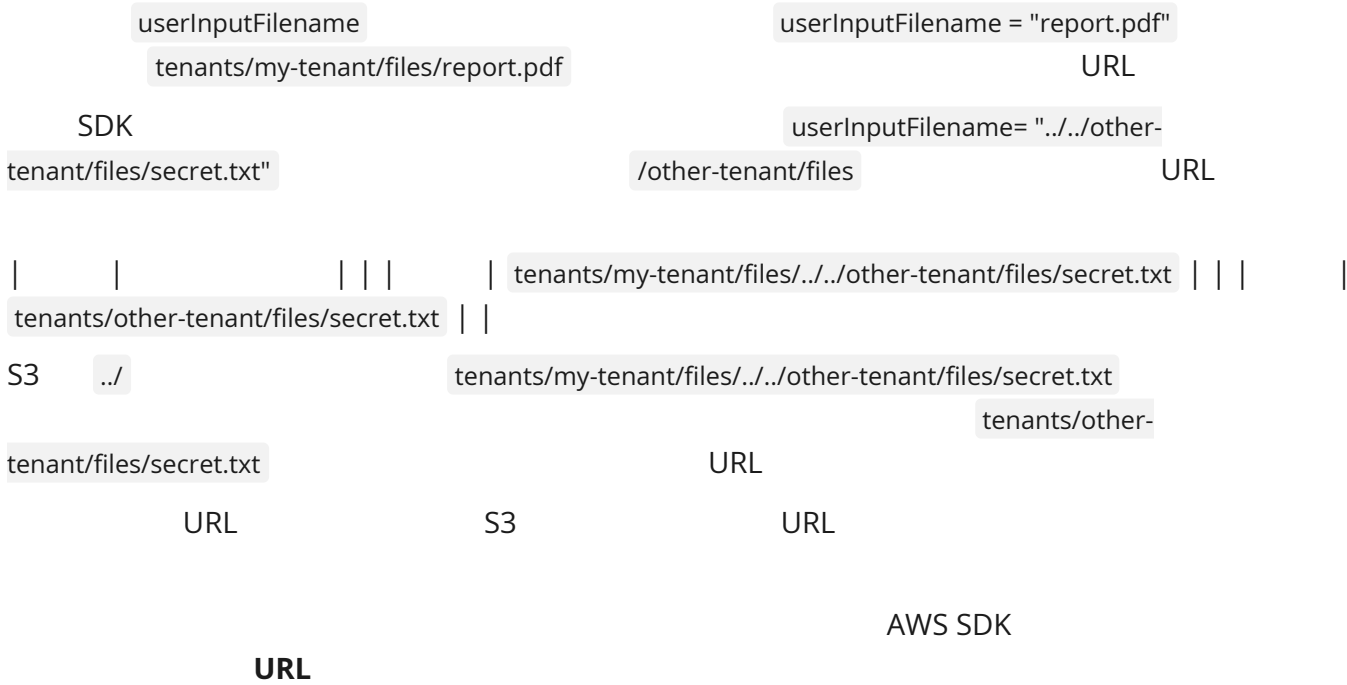
Amazon S3



```
func generatePresignedURL(tenantID, userInputFilename string) (string, error) {
    objectKey := fmt.Sprintf("tenants/%s/files/%s", tenantID, userInputFilename)

    req, _ := s3Client.GetObjectRequest(&s3.GetObjectInput{
        Bucket: aws.String("my-flatt-bucket"),
        Key:    aws.String(objectKey),
    })

    return req.Presign(15 * time.Minute)
}
```



AWS SDK

Go(v1) S3 URL

AWS SDK for Go(v1)

Go(v1)

URL

SDK

URL

URL

1. Request

URL

Request

GetObject

GetObjectRequest()

HTTPPath: "{Bucket}/{Key+}"

c.newRequest()

URL(: <https://s3.us-east-1.amazonaws.com>)

{Bucket}

{Key+}

aws-sdk-go-main/service/s3/api.go 4976~4990 v1.55.8

```
func (c *S3) GetObjectRequest(input *GetObjectInput) (req *request.Request, output *GetObjectOutput) {
    op := &request.Operation{
        Name:    opGetObject,
        HTTPMethod: "GET",
        HTTPPath: "{Bucket}/{Key+}",
    }

    if input == nil {
        input = &GetObjectInput{}
    }

    output = &GetObjectOutput{}
    req = c.newRequest(op, input, output)
    return
}
```

1. /

Presign()

Sign()

r.Build()

URL

aws-sdk-go-main/aws/request/request.go 436~447

```

func (r *Request) Sign() error {
    r.Build()
    if r.Error != nil {
        debugLogReqError(r, "Build Request", notRetrying, r.Error)
        return r.Error
    }

    SanitizeHostForHeader(r.HTTPRequest)

    r.Handlers.Sign.Run(r)
    return r.Error
}

```

URL

endpointHandler()

	URL	S3ForcePathStyle == true		S3ForcePathStyle == false		DNS	
	S3ForcePathStyle == false		DNS				
	S3ForcePathStyle	false		DNS		URL	
DNS		IP	: 192.0.2.1		..	HTTPS	
.		3	SDK Go(v1)		HTTPS	URL	
		URL					

aws-sdk-go-main/service/s3/endpoint.go 115~120

```

func endpointHandler(req *request.Request) {
    endpoint, ok := req.Params.(endpointARNGetter)
    if !ok || !endpoint.hasEndpointARN() {
        updateBucketEndpointFromParams(req)
        return
    }
}

```

aws-sdk-go-main/service/s3/host_style_bucket.go 36~49

```

func updateEndpointForS3Config(r *request.Request, bucketName string) {
    forceHostStyle := aws.BoolValue(r.Config.S3ForcePathStyle)
    accelerate := aws.BoolValue(r.Config.S3UseAccelerate)

    if accelerate && accelerateOpBlacklist.Continue(r) {
        if forceHostStyle {
            if r.Config.Logger != nil {
                r.Config.Logger.Log("ERROR: ...")
            }
        }
        updateEndpointForAccelerate(r, bucketName)
    } else if !forceHostStyle && r.Operation.Name != opGetBucketLocation {
        updateEndpointForHostStyle(r, bucketName)
    }
}

```

moveBucketToHost()

removeBucketFromPath()

/{Bucket}

aws-sdk-go-main/service/s3/host_style_bucket.go 133~137

```

func moveBucketToHost(u *url.URL, bucket string) {
    u.Host = bucket + "." + u.Host
    removeBucketFromPath(u)
}

```

1. path.Clean()

rest.Build()

buildURI()

aws-sdk-go-main/private/protocol/rest/build.go 201~216

```

func buildURI(u *url.URL, v reflect.Value, name string, tag reflect.StructTag) error {
    value, err := convertType(v, tag)

    u.Path = strings.Replace(u.Path, "{"+name+"}", value, -1)
    u.Path = strings.Replace(u.Path, "{"+name+"}", value, -1)

    u.RawPath = strings.Replace(u.RawPath, "{"+name+"}", EscapePath(value, true), -1)
    u.RawPath = strings.Replace(u.RawPath, "{"+name+"}", EscapePath(value, false), -1)

    return nil
}

```

`cleanPath()``cleanPath()``../``path.Clean()` Go

URL

aws-sdk-go-main/private/protocol/rest/build.go 137~139

```
if !aws.BoolValue(r.Config.DisableRestProtocolURICleaning) {  
    cleanPath(r.HTTPRequest.URL)  
}
```

aws-sdk-go-main/private/protocol/rest/build.go 247~258

```
func cleanPath(u *url.URL) {  
    hasSlash := strings.HasSuffix(u.Path, "/")  
  
    u.Path = path.Clean(u.Path)  
    u.RawPath = path.Clean(u.RawPath)  
  
    if hasSlash && !strings.HasSuffix(u.Path, "/") {  
        u.Path += "/"  
        u.RawPath += "/"  
    }  
}
```

1. URL

`Sign()``r.Handlers.Sign.Run(r)`

URL

aws-sdk-go-main/aws/request/request.go 436~447

```
func (r *Request) Sign() error {  
    r.Build()  
    if r.Error != nil {  
        debugLogReqError(r, "Build Request", notRetrying, r.Error)  
        return r.Error  
    }  
  
    SanitizeHostForHeader(r.HTTPRequest)  
  
    r.Handlers.Sign.Run(r)  
    return r.Error  
}
```

URL

`path.Clean()`

- URL

```
path.Clean()
```

bar

			endpointHandler()		{Bucket}/{Key+}		buildURI()	Bucket=my-flatt-
bucket, Key=../other-flatt-bucket/secret.txt					/my-flatt-bucket/../other-flatt-bucket/secret.txt			
path.Clean()					/other-flatt-bucket/secret.txt			

URL

URL

URL

IAM

●

```
path.Clean()
```

	(Host)	(Path)			endpointHandler()		my-flatt-bucket.s3.amazonaws.com				
	{Key+}			buildURI()	Key=foo/../../bar		my-flatt-bucket.s3.amazonaws.com		/foo/../../bar		

`path.Clean()` | `my-flatt-bucket.s3.amazonaws.com` | `/bar` | |

AWS

AWS

AWS

We are aware of many customers who have built up an implicit dependency on this behavior of path normalization. For that reason, we have retained this behavior by default for AWS SDK for Go V1 to maintain backward compatibility for customers that have older or difficult to update solutions that depend on this behavior.

AWS SDK for Go

V1

SDK `path.Clean()` AWS SDK for Go(v1)
`DisableRestProtocolURICleaning`
`../`
`aws-sdk-go-main/aws/config.go` 513-516

```
func (c *Config) WithDisableRestProtocolURICleaning(t bool) *Config {  
    c.DisableRestProtocolURICleaning = &t  
    return c  
}
```

AWS SDK for Go(v1)

Go(v1) Archive AWS SDK for Go(v2)

URL

JavaScript(v3) CloudFront URL

`@aws-sdk/cloudfront-signer` AWS SDK for JavaScript v3 CloudFront URL
JavaScript URL

1. `baseUrl`

URL `baseUrl` `getSignedUrl()` `url` `url`
`baseUrl` `policy` URL `baseUrl`

```

let baseUrl: string | undefined;
if (url) {
  baseUrl = url;
} else if (policy) {
  const resources = getPolicyResources(policy!);
  if (!resources[0]) {
    throw new Error(
      "@aws-sdk/cloudfront-signer: No URL provided and unable to determine URL from first policy statement resource."
    );
  }
  baseUrl = resources[0].replace("*/", "https://");
}

```

1. URL

new URL(baseUrl!) URL

```

const newURL = new URL(baseUrl!);
newURL.search = Array.from(newURL.searchParams.entries())
  .concat(Object.entries(cloudfrontSignBuilder.createCloudfrontAttribute()))
  .filter(([, value]) => value !== undefined)
  .map(([key, value]) => `${encodeURIComponent(key)}=${encodeURIComponent(value)}`)
  .join("&");

```

URL

../

URL

foo/../bar

URL

bar

URL

| | URL | | **getSignedUrl** url=<https://flatt-distribution.cloudfront.net/foo/../bar>
 r | <https://flatt-distribution.cloudfront.net/foo/../bar> | | **new URL()** | <https://flatt-distribution.cloudfront.net/bar> | |

CloudFront

URL

URL

CloudFront S3

URL S3

S3

../

CloudFront

S3

URL

AWS

AWS

I would like to inform you that our CNA team has evaluated your reported issue for a CVE/GHSA assignment and determined that this does not qualify for a CVE under our program [1] as this issue requires overly permissive S3 bucket policies to be applied. The configuration of these policies fall under the customer side of the AWS Shared Responsibility Model [2]. CVE/GHSA

CVE S3
AWS Shared Responsibility Model

sdk/cloudfront-signer v3.858.0 @aws-

@aws-sdk/cloudfront-signer v3.858.0 URL URL

aws-sdk-js-v3/packages/cloudfront-signer/src/sign.ts 139~146

```
const startFlag = baseUrl!.includes("?") ? "&" : "?";
const params = Object.entries(cloudfrontSignBuilder.createCloudfrontAttribute())
  .filter(([ , value]) => value !== undefined)
  .map(([key, value]) => `${encodeURIComponent(key)}=${encodeURIComponent(value)}`)
  .join("&");
const urlString = baseUrl + startFlag + params;

return getResource(urlString);
```

v3.858.0 @aws-sdk/cloudfront-signer

AWS SDK for JavaScript v3 @aws-sdk/cloudfront-signer
URL CloudFront S3
AWS

1

AWS SDK Go(v1) `path.Clean()`
URL

S3

- `path.normalize()` (JavaScript)
- `os.path.normpath()` (Python)
- `java.nio.file.Path.normalize()` (Java)

`path.normalize()`

```
const path = require("path");
const { S3Client, GetObjectCommand } = require("@aws-sdk/client-s3");
const { getSignedUrl } = require("@aws-sdk/s3-request-presigner");

const s3Client = new S3Client({ region: "ap-northeast-1" });

async function generatePresignedUrlWithNormalize(tenantId, userInputPath) {

  const rawPath = `tenants/${tenantId}/files/${userInputPath}`;

  const objectKey = path.normalize(rawPath);

  const command = new GetObjectCommand({
    Bucket: process.env.BUCKET_NAME,
    Key: objectKey,
  });

  return await getSignedUrl(s3Client, command, { expiresIn: 3600 });
}
```

2

// ./

../

- `path.join()` (JavaScript)
- `path.Join()` , `filepath.Join()` (Go)

`path.join()`

```

const path = require("path");
const { S3Client, GetObjectCommand } = require("@aws-sdk/client-s3");
const { getSignedUrl } = require("@aws-sdk/s3-request-presigner");

const s3Client = new S3Client({ region: "ap-northeast-1" });

async function generatePresignedUrlWithPathJoin(tenantId, userInputFilename) {

const objectKey = path.join("tenants", tenantId, "files", userInputFilename);

const command = new GetObjectCommand({
  Bucket: process.env.BUCKET_NAME,
  Key: objectKey,
});

return await getSignedUrl(s3Client, command, { expiresIn: 3600 });
}

```

URL

3	URL	URL	../	
	CloudFront	URL	URL	URL
			URL	

- `new URL()` (JavaScript)
- `java.net.URI.resolve()` (Java)
- `url.URL.ResolveReference()` (Go)

new URL()

```

const { getSignedUrl } = require("@aws-sdk/cloudfront-signer");

function generateCloudFrontSignedUrl(userInputPath) {

  const rawUrl = "https://d1111111abcdef8.cloudfront.net/tenants/my-tenant/files/" + userInputPath;

  const url = new URL(rawUrl);

  return getSignedUrl({
    url: url.toString(),
    keyPairId: "dummyKeyId",
    privateKey: dummyPrivateKey,
    dateLessThan: new Date(Date.now() + 3600 * 1000).toISOString(),
  });
}

```

	URL		AWS	SDK	Go(v1)	JavaScript(v3)
SDK						3
	URL					

GMO Flatt Security

Archived from https://blog.flatt.tech/entry/signed_url_path_traversal. Rendered offline from the local Markdown copy.